OntoTrial

Handbuch

Benutzung, Validierung der Extraktion
und technische Dokumentation

Teil I Benutzerhandbuch
Teil II Validierung: Prüfung, Referenz, KI-Judges, Regression, Studie, Modellvergleich
Teil III Technische Dokumentation

Stand 29. Oktober 2026, Schema-Version 3

Klinik für Augenheilkunde
Universitätsklinikum Freiburg

Inhaltsverzeichnis

I. Benutzerhandbuch **5**

1. Einführung **6**
1.1. Das Hauptfenster . . . 7

2. Kandidaten und Phenopackets **8**
2.1. Die Kandidatenliste . . . 8
2.2. Visual Profile . . . 8
2.3. Das Aktionsmenü . . . 9

3. Suchprogramm: Kohorten in natürlicher Sprache **10**

4. Klinische Studien **11**
4.1. Studie anlegen und Kriterien extrahieren . . . 11
4.2. Der Regel-Editor . . . 11

5. Matching **13**
5.1. Per Study . . . 13
5.2. Per Patient . . . 13

6. Time-to-Event **14**

7. Ontologie-Browser **15**

8. Werkzeuge **16**
8.1. Tagging . . . 16
8.2. Neu extrahieren und Matching nach Tag . . . 16
8.3. Propensity Matching . . . 16
8.4. Anonymisierter Export . . . 17
8.5. Long-Format-Export für die Statistik . . . 17

II. Validierung der Extraktion **18**

9. Validierung im Überblick **19**
9.1. Begriffe . . . 19
9.2. Ablauf auf einen Blick . . . 20
9.3. Rollen . . . 20

10. Das Validierungsfenster **21**
10.1. Öffnen . . . 21
10.2. Kopfzeile . . . 21
10.3. Die drei Reiter . . . 21

10.4. Hintergrundprozesse . . . 22

11. Datensätze anlegen und schützen 23
11.1. Neue Stichprobe ziehen . . . 23
11.2. Schutz des Datensatzes . . . 24
11.3. Datensatz löschen . . . 24

12. Anleitung für Prüferinnen und Prüfer 25
12.1. Vor dem Start . . . 25
12.2. Die Arbeitsfläche . . . 25
12.3. Ein Brief Schritt für Schritt . . . 26
12.4. Teil A: Checkliste (nur Studie) . . . 26
12.5. Teil B: Einträge bewerten . . . 26
12.6. Fehlende Einträge ergänzen . . . 27
12.7. Brief abschließen . . . 28
12.8. Bewertungsregeln (Kurzfassung) . . . 28
12.9. Häufige Fragen . . . 30

13. Doppelprüfung, Konsens und Referenz 31
13.1. Wie das System die Bewertungen eines Briefs zusammenführt . . . 31
13.2. Konsens . . . 31
13.3. Wie aus den Bewertungen die Referenz wird . . . 32
13.4. Einfrieren und Auftauen . . . 32

14. KI-Judges 33
14.1. Wozu . . . 33
14.2. Verfügbare Judges . . . 33
14.3. Bewerten lassen . . . 33
14.4. Beenden und aufräumen . . . 34
14.5. KI-Referenz einfrieren . . . 34
14.6. Datenschutz bei externen Judges . . . 34
14.7. Methodische Einordnung . . . 34

15. Regressionstests 35
15.1. Zwei Arten von Läufen . . . 35
15.2. Einrichten: Basis und Grundrauschen . . . 35
15.3. Im Alltag . . . 36
15.4. Die Kennzahlen . . . 36
15.5. Pipeline-Stand . . . 36

16. Validierungsstudie 37
16.1. Empfohlener Ablauf . . . 37
16.2. Die Kennzahlen im Reiter Studie . . . 37
16.3. Rohdaten für die Statistik . . . 38

17. Zwei Sprachmodelle vergleichen 39
17.1. Voraussetzungen . . . 39
17.2. Schritt für Schritt . . . 39
17.3. Ergebnisse festhalten . . . 40
17.4. Statistischer Vergleich in R . . . 41
17.5. Häufige Fallstricke . . . 41

III. Technische Dokumentation 43

18. Architektur 44
18.1. Dateien . . . 45

19. Extraktion und Mapping 46
19.1. Schritt 1: Zerlegung des Textes . . . 46
19.2. Schritt 2: Abbildung auf Terminologien . . . 46
19.3. Augenheilkunde . . . 47
19.4. Aufbau der Phenopackets . . . 48

20. Matching und Ähnlichkeit 49
20.1. Studien-Matching . . . 49
20.2. Phenopacket-Distanz . . . 49

21. Validierung: Algorithmen 50
21.1. Stichprobe . . . 50
21.2. Einträge und Schlüssel . . . 50
21.3. Zusammenführung und Referenz . . . 50
21.4. Vergleich einer Extraktion mit der Referenz . . . 50
21.5. Studienkennzahlen . . . 51
21.6. KI-Judge . . . 51

22. Datenbank 52
22.1. Schema und Migrationen . . . 52
22.2. Tabellen . . . 52
22.3. Schutz der Validierungsdaten . . . 53

23. Konfiguration 54
23.1. Sprachmodelle und Dienste . . . 54
23.2. Qualität des Mappings . . . 54
23.3. Import, Export, Validierung und KI-Judges . . . 55

24. Betrieb 56
24.1. Installation . . . 56
24.2. Prozesse und Neustarts . . . 56
24.3. Nützliche Befehle . . . 57
24.4. Terminologien aktualisieren . . . 57

25. API-Referenz 58
25.1. Extraktion . . . 58
25.2. Matching, Analyse und Export . . . 58
25.3. Validierung . . . 59
25.4. Live-Meldungen . . . 60

26. Sicherheit und Datenschutz 61

27. Fehlerbehebung 62

A. Kurzreferenz für Prüferinnen und Prüfer 63

Teil I.

Benutzerhandbuch

Kapitel 1.

Einführung

OntoTrial ist eine Arbeitsumgebung für die strukturierte Auswertung augenärztlicher Arztbriefe und klinischer Studienprotokolle. Sie verwandelt Freitext in ontologisch kodierte Daten:

- **Patienten** werden als *GA4GH Phenopackets v2* abgebildet: Diagnosen (ICD-10-GM), Befunde und Symptome (HPO), Eingriffe (OPS), Medikation (ATC) und Messwerte (LOINC), jeweils mit Augenseite, Datum und Status (vorhanden oder verneint).
- **Studien** werden als *FHIR R6 Group* mit verschachtelten Ein- und Ausschlusskriterien abgebildet.

Darauf aufbauend bietet OntoTrial Kohortensuche in natürlicher Sprache, Studien-Matching mit nachvollziehbarer Begründung, Zeit-bis-Eignung-Analysen, Ähnlichkeitssuche und Fall-Kontroll-Matching, anonymisierten Datenexport sowie ein Modul zur Validierung der Extraktion (Teil II).

> **Forschungsprototyp**
>
> OntoTrial ist kein Medizinprodukt und darf nicht für diagnostische oder therapeutische Entscheidungen verwendet werden. Patientendaten nur pseudonymisiert und im Rahmen von Datenschutz- und Ethikvorgaben verarbeiten. Daten verlassen die Einrichtung nur über den anonymisierten Export (Abschnitt 8.4).

1.1. Das Hauptfenster

Bereich	Funktion
Reiter oben	Candidates, Suchprogramm, Clinical Trials, Matching, Time-to-Event.
LLM Model	Sprachmodell für Extraktion, Studien-Extraktion und Zusammenfassungen, etwa `gpt-oss-120b` oder `glm-52`. Gilt für alle Aktionen, die danach gestartet werden.
Deep Mode	Zweistufige Extraktion: Das Modell analysiert den Text zunächst frei und füllt danach das strukturierte Schema. Gründlicher, aber langsamer.
Ontologie-Browser (rechts)	Suche und Navigation in HPO, ICD-10, OPS, ATC und LOINC (Kapitel 7).
Hintergrundprozesse	HUD-Panel unten rechts mit Fortschritt laufender Extraktionen, Matchings, Judges und Regressionsläufe.

Live-Aktualisierung. Alle geöffneten Browser sind über eine WebSocket-Verbindung mit dem Backend verbunden. Speichert jemand ein Phenopacket oder ändert einen Datensatz, erscheint die Änderung bei allen anderen automatisch. Es gibt keine Konfliktbehandlung: Bearbeiten zwei Personen dasselbe Feld gleichzeitig, gilt die letzte Speicherung.

Kapitel 2.

Kandidaten und Phenopackets

Ein *Kandidat* ist ein einzelner Brief eines Patienten. Mehrere Briefe desselben Patienten tragen dasselbe Pseudonym (PIZ).

2.1. Die Kandidatenliste

Element	Funktion
Suchfeld	Filtert nach Pseudonym, Tag oder ID.
Spalten	*ID*, *Pseudonym*, *Tags*, *Date* (Briefdatum).
+ und −	Kandidat anlegen bzw. ausgewählten Kandidaten entfernen.
Zahnrad-Menü	Werkzeuge für Gruppen von Kandidaten (Abschnitt 2.3).
Narrative Medical Record	Der Brieftext des ausgewählten Kandidaten.
Extract Phenopacket	Extrahiert das Phenopacket aus dem Brieftext mit dem gewählten Modell. Fortschritt im Panel Hintergrundprozesse; das Ergebnis wird am Kandidaten gespeichert.
Match against trials	Prüft den Kandidaten gegen alle Studien; Ergebnis im Reiter Matching.

2.2. Visual Profile

Der Reiter Visual Profile zeigt das Phenopacket als Tabelle mit den Spalten *Category*, *Code* und *Description & Modifiers*.

Kategorien
Demographics (Geschlecht, Alter), Measurement (LOINC-Messwerte), Phenotype (HPO), Disease/Diagnosis (ICD-10), Procedure (OPS), Medication (ATC).

Farbige Marken
Die erste Marke zeigt Code und Bezeichnung. Weitere Marken sind Modifikatoren: Augenseite (`HP:0012834` rechts, `HP:0012835` links, `HP:0012832` beidseits), Datum der Durchführung, Messwert mit Operator und Einheit.

Bearbeiten

+ fügt einen Eintrag hinzu, − entfernt den ausgewählten. Modifikatoren lassen sich per Doppelklick ändern. Änderungen werden ins Phenopacket zurückgeschrieben.

Der Reiter JSON Source zeigt dasselbe Phenopacket im Format GA4GH Phenopacket v2.

2.3. Das Aktionsmenü

Eintrag	Funktion
Tagging…	Tags für eine Liste von Pseudonymen setzen (Abschnitt 8.1).
Recompute Candidates with Tag…	Alle Kandidaten mit einem Tag im Hintergrund neu extrahieren, zum Beispiel nach einem Update der Pipeline.
Match tag against trials	Alle Kandidaten mit einem Tag gegen die Studien prüfen.
Propensity Matching…	Fall-Kontroll-Matching nach Phenopacket-Ähnlichkeit (Abschnitt 8.3).
Validation…	Validierungsfenster (Teil II).
Anonymized Export…	Anonymisierte Phenopackets für die Weitergabe (Abschnitt 8.4).

Kapitel 3.

Suchprogramm: Kohorten in natürlicher Sprache

Im Reiter Suchprogramm beschreiben Sie eine Kohorte in eigenen Worten, etwa „Patienten mit Epitheldefekt bei neurotropher Keratopathie, aber ohne perforierende Keratoplastik“.

1. Anfrage in das Feld *Ihre Kohorten-Anfrage* schreiben und Senden klicken.
2. Ein Assistent übersetzt die Begriffe mit derselben Mapping-Pipeline wie die Extraktion in Codes und erzeugt eine SQL-Abfrage. Sie erscheint im Feld *PostgreSQL / JSONB Code*. Hierarchien werden berücksichtigt: Ein Oberbegriff findet auch alle Unterbegriffe. Verneinte Befunde zählen nicht als Treffer.
3. Die gefundenen Patienten stehen unter *Gefundene Patienten (Kohorte)*.
4. Die SQL-Abfrage lässt sich von Hand anpassen und mit SQL ausführen erneut starten. Erlaubt sind nur lesende Abfragen (`SELECT`, `WITH`).
5. Kohorte taggen versieht alle gefundenen Patienten mit einem Tag, Pseudonyme kopieren kopiert die Liste in die Zwischenablage.

> **Tipp**
> Prüfen Sie die erzeugte SQL-Abfrage, bevor Sie eine Kohorte weiterverwenden. Sie zeigt genau, welche Codes gesucht werden.

Kapitel 4.

Klinische Studien

Der Reiter Clinical Trials verwaltet Studien und ihre Ein- und Ausschlusskriterien.

4.1. Studie anlegen und Kriterien extrahieren

1. Mit + unter der Studienliste eine Studie anlegen.
2. Den Protokolltext (Synopse, Ein- und Ausschlusskriterien) in *Study Synopsis / Protocol Context* einfügen.
3. Extract FHIR Group: Das Sprachmodell zerlegt das Protokoll in einzelne Kriterien und kodiert sie. Das Ergebnis erscheint im Regel-Editor.
4. Kriterien prüfen und korrigieren.

4.2. Der Regel-Editor

Jede Zeile ist ein Kriterium oder eine Gruppe:

Kriterium
Typ (Diagnosis, Symptom/Phenotype, Procedure, Medication, LOINC/Lab Assay, Demographic), Bedingung (All must be present für Einschluss, Neither must be present für Ausschluss) und eine Marke mit Code und Bezeichnung, bei Messwerten mit Grenzwert, etwa „Age: ≥ 18 years“.

Gruppe
All, Any oder Neither of the following are true; Gruppen lassen sich verschachteln.

Knöpfe
Add Criterion, Add Group, Delete Criterion, Reset; View FHIR R6 JSON zeigt die Studie als FHIR-Ressource.

Codes lassen sich aus dem Ontologie-Browser übernehmen: Begriff suchen und über das Feld *Drag from here* in die Marke eines Kriteriums ziehen.

Hierarchie

Ein Kriterium mit einem Oberbegriff, etwa OPS 5-125 (Keratoplastik), ist auch durch alle Unterbegriffe erfüllt. Für eine engere Auswahl den spezifischen Code wählen.

Kapitel 5.

Matching

5.1. Per Study

1. Studie in der Liste wählen.
2. Run Matching for Selected Trial prüft alle Kandidaten. Mit *Filter Candidate* lässt sich die Ergebnisliste eingrenzen.
3. Die Ergebnisliste zeigt je Kandidat *Eligibility Status* und Zeitpunkt.
4. Ein Klick auf eine Zeile zeigt die Begründung Kriterium für Kriterium.

Match Status	Bedeutung
Satisfied	Einschlusskriterium erfüllt; die Spalte *Evidence* nennt den Beleg.
Missing (Unmet)	Einschlusskriterium nicht belegt.
Clear (No Info)	Ausschlusskriterium nicht vorhanden.
Violation	Ausschlusskriterium vorhanden; der Patient ist nicht geeignet.

Ausdrücklich verneinte Befunde gelten als Beleg für das Fehlen. Zeitliche Bedingungen werden gegen das Datum des Eingriffs oder der Medikation in derselben Gruppe geprüft. Create Narrative Summary lässt das Sprachmodell eine Begründung in Worten formulieren.

Studienauge. Viele Studien unterscheiden Studienauge und Partnerauge. OntoTrial prüft deshalb zwei Hypothesen: rechtes Auge als Studienauge und linkes Auge als Studienauge (mit gespiegeltem Phenopacket). Das Ergebnis nennt, über welches Auge der Patient geeignet ist.

5.2. Per Patient

Im Unterreiter Per Patient wird umgekehrt ein Kandidat gegen alle Studien geprüft, mit derselben Begründung je Kriterium.

Kapitel 6.

Time-to-Event

Für eine gewählte Studie berechnet `Calculate Time to Eligibility`, ab welchem Brief ein Patient erstmals die Kriterien erfüllt. Dazu werden alle Briefe je Pseudonym chronologisch ausgewertet.

Spalte	Bedeutung
Event (0/1)	1, wenn die Kriterien irgendwann erfüllt waren; 0 = zensiert.
Time (Days)	Tage vom ersten Brief (Baseline) bis zum Ereignis bzw. bis zum letzten Brief.
Study Eye	Auge, über das die Eignung besteht.
Baseline Date, *Event Date*	Datum des ersten Briefs bzw. des Ereignisses.

`Export CSV for R` lädt die Tabelle für eine Überlebenszeitanalyse in R, Python oder SPSS herunter.

Kapitel 7.

Ontologie-Browser

Rechts im Hauptfenster stehen die fünf Terminologien als Reiter: HPO, ICD10, OPS, ATC, LOINC.

- **Suche:** Begriff oder Code eingeben. Die Treffer zeigen den Pfad in der Hierarchie. *Name only* beschränkt die Suche auf Bezeichnungen und lässt Synonyme außen vor.
- **Hierarchie:** Unter *Hierarchy Nodes* lässt sich der Baum aufklappen.
- **Details:** Definition, Synonyms und Cross References des gewählten Begriffs.
- **Übernehmen:** Über *Drag from here* wird der gewählte Begriff in den Regel-Editor oder ein Profil gezogen.

Kapitel 8.

Werkzeuge

8.1. Tagging

Tags fassen Kandidaten zu Kohorten zusammen. Im Fenster Kandidaten-Tagging (Batch) eine Liste von Pseudonymen einfügen (durch Komma, Leerzeichen oder Zeilen getrennt), den Tag-Namen angeben und Tag setzen klicken. Mit *Bestehende Tags beibehalten* wird der Tag angehängt; ohne Haken ersetzt er die bisherigen Tags. Tags lassen sich außerdem aus dem Suchprogramm und dem Propensity Matching heraus setzen.

8.2. Neu extrahieren und Matching nach Tag

Recompute Candidates with Tag… zeigt zunächst, wie viele Kandidaten betroffen sind, und extrahiert sie dann im Hintergrund mit dem aktuellen Stand der Pipeline neu. Match tag against trials prüft alle Kandidaten mit dem Tag gegen die Studien.

8.3. Propensity Matching

Das Fenster Propensity Matching (Phenopacket-Distanz) sucht zu jedem Fall die ähnlichsten Kontrollen.

Feld	Bedeutung
Fälle (Tag)	Tag der Fallgruppe.
Kontrollpool	Tag der möglichen Kontrollen; leer = alle Kandidaten.
Verhältnis 1:	Anzahl Kontrollen je Fall.
Caliper	Höchste zulässige Distanz (0 bis 1).
Geschlecht exakt	Nur Kontrollen gleichen Geschlechts.
RA = LA (ein-/beidseitig)	Rechtes und linkes Auge gelten als gleichwertig; unterschieden wird nur ein- gegenüber beidseitig.
Kontrollen mehrfach verwenden	Eine Kontrolle darf mehreren Fällen zugeordnet werden.
Gewichte	Anteil der Merkmalsblöcke an der Distanz: Demografie, Diagnosen, Phänotypen, Prozeduren, Medikation, Messwerte.

Matching starten zeigt die Paare mit Distanz. Kontrollen taggen versieht die gefundenen Kontrollen mit einem Tag, CSV exportieren lädt die Paare herunter. Die Berechnung der Distanz beschreibt Abschnitt 20.2.

8.4. Anonymisierter Export

Anonymized Export... erzeugt für eine Kohorte (Tag oder Liste) de-identifizierte Phenopackets, eines je Brief:

1. **Kein Freitext:** Nur geprüfte Codes mit ihrer Bezeichnung aus der Terminologie.
2. **Entkoppelte Kennungen:** Zufällige IDs je Exportlauf; wiederholte Exporte lassen sich nicht verknüpfen; die Reihenfolge wird gemischt.
3. **Datumsverschiebung:** Daten auf Monate gekürzt und je Patient um ±1 bis 3 Monate verschoben; Abstände bleiben erhalten.
4. **Seltene Codes:** Alter ab 90 zusammengefasst; Codes werden entlang der Hierarchie verallgemeinert, bis jeder bei mindestens k Patienten vorkommt ($k \geq 5$).

> **Grenzen**
>
> k-Anonymität gilt je Code, nicht für die Kombination aller Codes eines Patienten. Ob ein Export für den Empfänger anonym ist, entscheidet der Datenschutz.

8.5. Long-Format-Export für die Statistik

Für eigene Analysen liefert das Backend alle kodierten Elemente in einer Zeile je Element (`/BBB/export/long/events`) und eine Zeile je Brief (`/BBB/export/long/patients`), als CSV oder NDJSON, direkt lesbar in R. Filter, Spalten und ein R-Beispiel stehen in Kapitel 25. Der Export enthält Pseudonyme und exakte Daten und ist nur für interne Analysen gedacht.

Teil II.

Validierung der Extraktion

Kapitel 9.

Validierung im Überblick

Das Validierungsmodul prüft, wie gut OntoTrial Arztbriefe in Phenopackets übersetzt. Es beantwortet zwei Fragen:

Ist eine Änderung an der Pipeline unbedenklich? (Regressionstest)
Nach jedem Update an Prompts, Intercepts, Modell oder Code laufen die Briefe eines festen Referenzdatensatzes erneut durch die Pipeline. Das Ergebnis wird mit der Referenz und mit einem früheren Basis-Lauf verglichen. Verschlechtert sich etwas, ist das sofort sichtbar, bevor die Änderung den ganzen Bestand betrifft.

Wie gut ist die Extraktion? (Validierungsstudie)
Ärztinnen und Ärzte bewerten eine repräsentative Stichprobe nach festen Regeln. Daraus entstehen Kennzahlen mit Konfidenzintervallen, die sich publizieren lassen: Richtigkeit je Domäne, Genauigkeit von Seite und Datum, Vollständigkeit, Übereinstimmung der Prüfer.

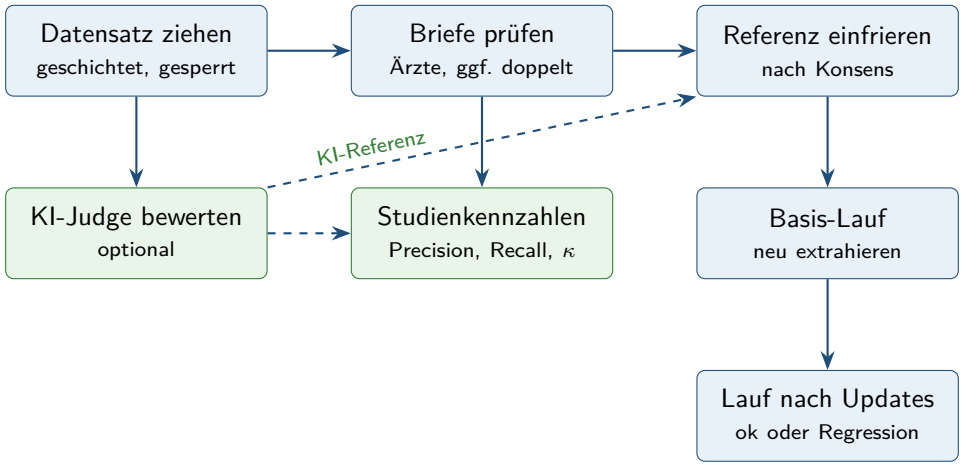
Beide Fragen nutzen dieselben Bausteine. Ein ärztlich geprüfter Studiendatensatz taugt anschließend auch als Referenz für Regressionstests.

9.1. Begriffe

Begriff	Bedeutung
Datensatz	Stichprobe von Briefen aus dem Bestand, mit Kopie von Brieftext und extrahiertem Phenopacket zum Zeitpunkt der Ziehung. Zweck *Regressionstest* oder *Validierungsstudie*.
Eintrag	Ein einzelnes kodiertes Element eines Phenopackets, zum Beispiel „Pseudophakie, ICD-10 Z96.1, LA“.
Bewertung	Urteil einer Prüferin oder eines Prüfers zu einem Eintrag (korrekt oder Fehlertyp), dazu ergänzte fehlende Einträge und bei Studien die Checkliste.
Konsens	Prüfer mit dem festen Namen Konsens. Klärt Abweichungen zwischen zwei Prüfern und sieht dabei die Urteile beider.
Referenz	Aus den Bewertungen abgeleiteter Sollzustand je Brief. Wird *eingefroren* und ist dann unveränderlich.
KI-Judge	Sprachmodell, das die Einträge nach denselben Regeln bewertet wie die Ärzte. Seine Urteile werden getrennt gespeichert.

Begriff	Bedeutung
KI-Referenz	Referenz, die aus den Urteilen eines KI-Judge statt aus ärztlichen Bewertungen entsteht.
Lauf	Vergleich der Extraktion aller Briefe eines Datensatzes mit der Referenz; entweder neu extrahiert oder mit den gespeicherten Phenopackets.
Basis	Ein ausgewählter Lauf, mit dem spätere Läufe verglichen werden.
Pipeline-Stand	Fingerabdruck von Backend, Modell, Prompts, Intercepts, Filterregeln und Mapping-Schaltern, der mit jedem Lauf gespeichert wird.

9.2. Ablauf auf einen Blick



9.3. Rollen

Rolle	Aufgaben
Koordination	Datensätze anlegen, Pilotrunde organisieren, Referenz einfrieren, Sicherungen laden, Kennzahlen auswerten.
Prüferinnen, Prüfer	Briefe bewerten (Kapitel 12). Sehen nur ihre eigenen Bewertungen.
Konsens	Abweichungen zwischen zwei Prüfern klären (Abschnitt 13.2).
Betrieb	Backend, Minion-Worker, Konfiguration und KI-Judges (Kapitel 24).

Kapitel 10.

Das Validierungsfenster

10.1. Öffnen

In der Kandidatenliste öffnet das Aktionsmenü der Knopfleiste (Zahnrad) den Eintrag Validation…. Das Fenster passt sich beim Öffnen an die Größe des Browserfensters an und lässt sich mit der Maus vergrößern; Brieftext und Tabellen wachsen mit.

10.2. Kopfzeile

Element	Funktion
Datensatz	Auswahl des Datensatzes; in Klammern der Zweck.
Neuer Datensatz …	Neue Stichprobe ziehen (Kapitel 11).
Sicherung laden	Lädt den kompletten Datensatz mit allen Bewertungen und Läufen als JSON-Datei herunter.
Prüfer	Ihr Name oder Kürzel. Alle Bewertungen werden unter diesem Namen gespeichert; der Browser merkt sich den Eintrag.
Infozeile	Anzahl der Briefe, Seed, Sperrstatus und Stand der Referenz.

10.3. Die drei Reiter

Prüfen
Briefe Eintrag für Eintrag bewerten. Arbeitsplatz der Prüferinnen und Prüfer.

Referenz & Regression
Stand der Prüfung, Referenz einfrieren, KI-Judges starten, Regressionsläufe starten und auswerten.

Studie
Kennzahlen nach Bewertungshandbuch berechnen und Rohdaten exportieren.

10.4. Hintergrundprozesse

KI-Judges und Regressionsläufe arbeiten im Hintergrund. Ihr Fortschritt erscheint im Panel OntoTrial Hintergrundprozesse, zum Beispiel „Brief 20 (29 s) · 19 bewertet, 0 Fehler“, und zusätzlich oben im Statusfeld des Reiters Referenz & Regression. Das Fenster darf währenddessen geschlossen werden; der Lauf geht weiter.

Kapitel 11.

Datensätze anlegen und schützen

11.1. Neue Stichprobe ziehen

Neuer Datensatz ... öffnet einen Dialog:

Feld	Bedeutung
Name	Eindeutiger Name, zum Beispiel `referenz_2026_1`. Wird später zum Bestätigen geschützter Aktionen gebraucht.
Zweck	*Regressionstest* oder *Validierungsstudie*. Nur Studien haben Checkliste, Doppelprüfung und eingestreute Fehler.
Anzahl Briefe	Vorschlag 50 (Regression) bzw. 150 (Studie); erlaubt 5 bis 2000.
Seed	Startwert der Zufallsziehung. Mit gleichem Seed und gleichem Bestand entsteht dieselbe Stichprobe. Leer = zufällig.
Nur Tag	Optional: nur Briefe von Patienten mit diesem Tag, etwa einer Kohorte.
höchstens ein Brief je Patient	Standard. Verhindert, dass ein Patient mit vielen Briefen das Ergebnis dominiert.
Doppelt prüfen (Anteil)	Nur Studie, Standard 0,3: Anteil der Briefe, die zwei Prüfer unabhängig bewerten.
Eingestreute Fehler	Nur Studie, Standard 0,08: Anteil der Briefe mit einem absichtlich eingebauten Fehler.
Zum Bestätigen ANLEGEN	Schutz vor versehentlichem Anlegen: `ANLEGEN` eintippen.

So wird gezogen. In Frage kommen Briefe mit Text (mindestens 300 Zeichen) und Phenopacket, die noch in keinem anderen Validierungsdatensatz stecken. Identische Brieftexte zählen einmal. Die Stichprobe wird proportional nach Quelle und Jahr geschichtet, sodass jede Schicht etwa so stark vertreten ist wie im Bestand. Die Reihenfolge der Briefe im Datensatz ist zufällig.

Eingestreute Fehler. Bei Studien-Datensätzen wird in den ausgewählten Briefen genau ein Eintrag manipuliert, bevor die Prüfer ihn sehen: die Augenseite vertauscht, das Datum um ein Jahr verschoben oder ein Eintrag entfernt. Welche Briefe betroffen sind, zeigt die Oberfläche nicht an. Für die Referenz zählt immer der unveränderte Eintrag; die Manipulation misst nur, wie aufmerksam geprüft wird.

11.2. Schutz des Datensatzes

Ein validierter Datensatz ist wertvoll. Deshalb gilt:

- Brieftext und Extraktion werden bei der Ziehung kopiert. Neu-Extraktionen oder Löschungen im Bestand verändern den Datensatz nicht.
- Jeder neue Datensatz ist **gesperrt**. Ein gesperrter Datensatz lässt sich nicht löschen.
- Nach dem **Einfrieren** der Referenz sind Bewertungen unveränderlich.
- Diese Regeln erzwingt die Datenbank selbst über Trigger, also auch gegen direkte SQL-Befehle. Die allgemeinen Tabellen-Routen der API sind für die Validierungstabellen gesperrt.
- Entsperren, Einfrieren, Auftauen und Löschen verlangen das Eintippen des Datensatznamens.

> **Sicherung**
> Vor jedem größeren Schritt, insbesondere vor Entsperren oder Löschen, mit `Sicherung laden` eine JSON-Kopie herunterladen. Sie enthält Briefe, Extraktionen, alle Bewertungen und Läufe.

11.3. Datensatz löschen

Im Reiter Referenz & Regression: erst `Sperren / Entsperren …`, dann `Datensatz löschen …`. Beides mit dem Namen bestätigen. Eine eingefrorene Referenz wird dabei mitgelöscht.

Kapitel 12.

Anleitung für Prüferinnen und Prüfer

12.1. Vor dem Start

1. Oben im Feld *Prüfer* den eigenen Namen oder ein festes Kürzel eintragen und mit Enter bestätigen. Immer dasselbe Kürzel verwenden, sonst zählt das System zwei verschiedene Prüfer.
2. Den richtigen Datensatz auswählen.
3. Das Bewertungshandbuch kennen; die Kurzfassung steht in Abschnitt 12.8.
4. Bei Studien: an der Pilotrunde teilnehmen, bevor die Hauptrunde beginnt.

> **Grundsatz**
> Referenz ist ausschließlich der **Brieftext**. Bewertet wird, ob die Extraktion den Brief richtig wiedergibt, nicht ob der Brief medizinisch stimmt. Akte, Bildgebung und eigenes Wissen bleiben außen vor.

12.2. Die Arbeitsfläche

Links: Briefliste
Nummer, Pseudonym und Stand: offen, 5/23 (5 von 23 Einträgen bewertet), Checkliste ✓ oder fertig. Der Zusatz 2× markiert Briefe, die doppelt geprüft werden. Darunter der eigene Fortschritt.

Mitte: Brieftext
Der vollständige Brief, wie er der Extraktion vorlag.

Rechts oben: Teil A
Nur bei Studien: die Checkliste.

Rechts: Teil B
Die extrahierten Einträge mit Domäne, Bezeichnung, Code, Seite, Datum, Wert und Ihrem Urteil.

Rechts unten
Fehlende Einträge sowie Brief abschließen und Nächster offener Brief.

12.3. Ein Brief Schritt für Schritt

1. Brief in der Liste anklicken und den Text vollständig lesen.
2. *Nur bei Studien:* Teil A, die Checkliste, ausfüllen, solange die Extraktion noch verborgen ist (Abschnitt 12.4).
3. Teil B: jeden Eintrag bewerten (Abschnitt 12.5).
4. Fehlende Einträge ergänzen (Abschnitt 12.6).
5. *Nur bei Studien:* für jede Checklistenzeile den Abgleich setzen.
6. Brief abschließen. Das System springt zum nächsten offenen Brief.

Jede Eingabe wird sofort gespeichert. Sie können jederzeit unterbrechen und später weitermachen.

12.4. Teil A: Checkliste (nur Studie)

Die Checkliste misst die Vollständigkeit unverzerrt. Deshalb ist die Extraktion verborgen, bis die Checkliste abgeschlossen ist.

1. Zeile hinzufügen … öffnet einen Dialog mit *Art* (Augen, Diagnose, Eingriff, Medikation, Visus, Tensio, Messwert), *Bezeichnung*, *Augenseite*, *Datum* und *Wert*. Stichworte genügen, Codes sind nicht nötig.
2. Erfasst werden: betroffene Augen, Hauptdiagnosen aus dem Diagnosenblock, alle Eingriffe mit Seite und Datum (auch anamnestische, mit Vermerk „geplant“, falls geplant), Augenmedikation, letzter Visus und Augendruck je Auge sowie Endothelzelldichte, Hornhautdicke oder Refraktion, falls genannt.
3. Checkliste abschließen, Extraktion zeigen. Danach erscheint Teil B. Die Checkliste lässt sich nur noch mit Rückfrage ändern.
4. Nach der Bewertung von Teil B jede Checklistenzeile markieren und Gefunden, Attributfehler (gefunden, aber Seite, Datum oder Wert falsch oder fehlend) oder Fehlt klicken.

12.5. Teil B: Einträge bewerten

Eintrag in der Tabelle markieren, dann einen der Knöpfe klicken. Nach jedem Urteil springt die Auswahl zum nächsten Eintrag.

Knopf	Wann
Korrekt	Konzept, Status, Seite, Datum und Wert stimmen.
Fehler …	Irgendetwas stimmt nicht. Öffnet den Fehler-Dialog; ebenso ein Doppelklick auf den Eintrag.
Dublette	Zweiter Eintrag zum selben Befund mit gleicher Seite. Zählt nicht als Fehler.
Unklar	Der Brief lässt keine Entscheidung zu. Ein Kommentar ist Pflicht.
Zurücksetzen	Urteil zurücknehmen.
Übrige korrekt	Alle noch nicht bewerteten Einträge als korrekt markieren (mit Rückfrage). Nur verwenden, wenn wirklich alle übrigen Einträge geprüft sind.

Der Fehler-Dialog

Jedes Merkmal wird getrennt bewertet. So lässt sich später berichten, ob der Code selbst oder nur ein Attribut falsch ist.

Merkmal	Urteile	Erläuterung
Konzept	korrekt · zu allgemein · zu spezifisch · falsches Konzept · nicht im Brief · falscher Bezug	„Falscher Bezug“: betrifft nicht den Patienten (Familie, Spender). „Zu allgemein“: richtige Oberklasse, der Brief stützt einen genaueren Code.
Status	korrekt · Verneinung falsch · Verdacht als gesichert · Geplantes als erfolgt	Bezieht sich auf vorhanden/verneint. Verneinte Einträge sind in der Tabelle mit [verneint] markiert.
Augenseite	korrekt · falsch · fehlt · überflüssig · nicht anwendbar	Daneben *richtig*: RA, LA, beidseits oder keine Seite. Bitte immer angeben, wenn bekannt.
Datum	dieselben Urteile	Daneben *richtig*: Datum so genau wie im Brief (JJJJ, JJJJ-MM oder JJJJ-MM-TT).
Messwert	korrekt · falsch · nicht anwendbar	Nur bei Messwerten; daneben der richtige Wert.
Kommentar	frei	Kurz begründen, besonders bei Grenzfällen.

> **Warum die richtige Angabe wichtig ist**
>
> Aus „Seite falsch, richtig: LA“ wird in der Referenz ein Eintrag mit LA. Fehlt die richtige Angabe, kann die Seite in späteren Regressionstests nicht geprüft werden.

12.6. Fehlende Einträge ergänzen

Steht etwas im Brief, das nicht extrahiert wurde, Fehlenden Eintrag ergänzen … klicken:

1. *Art* (Domäne) und *Bezeichnung* wie im Brief eintragen.

2. Code suchen schlägt über die Mapping-Pipeline einen Code vor; die Bezeichnung aus der Terminologie kann übernommen werden. Alternativ den Code von Hand eintragen, etwa `ICD10:H18.5`.
3. Augenseite, Datum, Wert, Einheit und *im Brief verneint* ergänzen.
4. Übernehmen.

Ohne Code zählt der Eintrag in der Studie, aber nicht im Regressionstest. Versehentlich ergänzte Einträge mit Entfernen wieder streichen.

12.7. Brief abschließen

Brief abschließen ist erst möglich, wenn alle Einträge ein Urteil haben und bei Studien jede Checklistenzeile abgeglichen ist. Ein abgeschlossener Brief lässt sich mit Wieder öffnen erneut bearbeiten, solange die Referenz nicht eingefroren ist. Nächster offener Brief springt zum nächsten Brief, den Sie noch nicht abgeschlossen haben.

12.8. Bewertungsregeln (Kurzfassung)

Allgemeine Regeln

- **Anamnestische Angaben** („Z. n.“, „bekannte“, „seit“) gehören zum Brief. Ihr Datum muss der Genauigkeit im Brief entsprechen.
- **Verneinte Befunde** („kein Anhalt für“, „keine“, „ausgeschlossen“) sind korrekt, wenn sie als verneint extrahiert sind; als vorhanden extrahiert ist das ein Statusfehler.
- **Verdacht und Differenzialdiagnose** („V. a.“, „DD“, „fraglich“) als gesichert extrahiert: Statusfehler. Nicht extrahiert ist kein Fehlen.
- **Geplante Maßnahmen** („geplant“, „empfohlen“, „Vorstellung zur“) als erfolgt extrahiert: Statusfehler.
- **Fremder Bezug**: Familienanamnese, Spenderangaben, andere Personen gehören nicht zum Patienten.
- **Zitierte Vorbefunde** anderer Kliniken gehören zum Brief.
- **Aktuelle Befunde ohne eigenes Datum** sind mit Brief- oder Untersuchungsdatum korrekt datiert. Ein anamnestisches Ereignis ohne Datum darf das Briefdatum *nicht* tragen (Datum überflüssig).
- **Messwertpaare** wie „Tensio 18/21 mmHg“ werden als RA/LA gelesen.
- **Im Zweifel** Unklar mit Kommentar statt raten.

Regeln je Domäne

Domäne	Korrekt, wenn	Häufige Fehler
Diagnose (ICD-10)	Code bezeichnet die genannte Krankheit; spezifischster Code, den der Brief stützt. Z-Codes für Zustände nach Eingriff sind zulässig.	Oberklasse statt Endcode; Verdacht als gesichert.
Befund (HPO)	Term trifft den Befund; bedeutungsgleicher Nachbarterm ist korrekt.	Sehr allgemeiner Term wie „Abnormality of the eye“.
Eingriff (OPS)	Code entspricht dem durchgeführten Eingriff.	Geplanter Eingriff als erfolgt; Briefdatum statt OP-Datum.
Medikation (ATC)	Wirkstoff stimmt; Kombinationspräparate als Kombinationscode oder Einzelwirkstoffe. Dosis und Frequenz werden nicht bewertet.	Präparat falsch aufgelöst; Seite fehlt bei „RA“, „LA“, „bds.“.
Messwert (LOINC)	Messgröße, Zahl, Einheit, Operator und Auge stimmen; Visus dezimal, Fingerzählen o. ä. als Text.	Werte zwischen RA und LA vertauscht; Lochblende als Hauptvisus.

Grenzfälle

Brieftext	Extraktion	Urteil	Begründung
„Z. n. DMEK RA 03/2021“	OPS 5-125.*, RA, 2021-03	korrekt	Datum in der Genauigkeit des Briefs
„Z. n. DMEK RA“ (ohne Datum)	OPS 5-125.*, RA, Briefdatum	Datum überflüssig	anamnestisch, kein Briefdatum
„V. a. Transplantatabstoßung LA“	T86.83, LA, vorhanden	Status: Verdacht als gesichert	Verdacht ist nicht gesichert
„Kein Anhalt für Makulaödem“	HPO Makulaödem, verneint	korrekt	Verneinung richtig
„Visus RA 0,5 cc, LA FZ“	RA 0,5; LA Text „FZ“	korrekt	nicht numerischer Visus als Text
„Tensio 18/21 mmHg“	RA 18, LA 21	korrekt	Konvention RA/LA
„Dexa-Gentamicin AT bds.“	ATC S01CA01, beidseits	korrekt	Kombinationscode zulässig
„Fuchs-Endotheldystrophie“	ICD-10 H18	Konzept zu allgemein	Brief stützt H18.5
„Pseudophakie RA, LA phak“	Z96.1, beidseits	Seite falsch, richtig RA	nur RA pseudophak

Brieftext	Extraktion	Urteil	Begründung
„DMEK RA geplant 11/2026“	OPS 5-125.*, erfolgt	Status: Geplantes als erfolgt	noch nicht durchgeführt
„Mutter mit Glaukom“	ICD-10 H40, vorhanden	Konzept: falscher Bezug	Familienanamnese

12.9. Häufige Fragen

Ein Eintrag hat einen richtigen Code, aber die falsche Seite. Ist das „Korrekt“ oder „Fehler“?
Fehler …, Konzept *korrekt*, Augenseite *falsch*, richtige Seite angeben.

Derselbe Befund ist zweimal extrahiert.
Den ersten bewerten, den zweiten als Dublette.

Der Brief nennt nur das Jahr, die Extraktion ein genaues Datum.
Nach der Regel der Pilotrunde verfahren; im Zweifel Unklar mit Kommentar.

Ich habe mich verklickt.
Eintrag markieren und neu bewerten oder Zurücksetzen. Ergänzte Einträge mit Entfernen streichen.

Die Knöpfe reagieren nicht mehr.
Wahrscheinlich ist die Referenz eingefroren. Dann sind keine Änderungen mehr möglich; die Meldung sagt das.

Kapitel 13.

Doppelprüfung, Konsens und Referenz

13.1. Wie das System die Bewertungen eines Briefs zusammenführt

1. Hat der Prüfer Konsens den Brief abgeschlossen, gilt sein Urteil.
2. Sonst: Ist der Brief einfach zu prüfen und von einem Prüfer abgeschlossen, gilt dessen Urteil.
3. Ist der Brief doppelt zu prüfen, braucht es zwei abgeschlossene Prüfungen. Stimmen beide in allen Einträgen und in den ergänzten fehlenden Einträgen überein, gilt das gemeinsame Urteil.
4. Weichen sie ab, ist der Brief eine *Abweichung* und braucht den Konsens.

13.2. Konsens

1. Im Reiter Referenz & Regression Stand aktualisieren: die Statusanzeige nennt die Briefe mit Abweichungen.
2. Im Feld *Prüfer* den Namen `Konsens` eintragen (genau so geschrieben).
3. Im Reiter Prüfen erscheint die zusätzliche Spalte *Andere Prüfer* mit den Urteilen beider Prüfer.
4. Jeden betroffenen Brief gemeinsam vollständig bewerten und abschließen.

Die ursprünglichen Einzelurteile bleiben gespeichert und gehen in die Übereinstimmungsanalyse (κ) ein.

13.3. Wie aus den Bewertungen die Referenz wird

Urteil	Folge für die Referenz
korrekt	Eintrag wird übernommen.
Seite, Datum oder Wert falsch/fehlend	Eintrag mit der richtigen Angabe; ohne richtige Angabe wird dieses Attribut nicht geprüft.
überflüssige Seite oder Datum	Eintrag ohne diese Angabe; eine spätere Angabe gilt als Fehler.
Verneinung falsch	Status umgedreht übernommen.
Verdacht als gesichert, Geplantes als erfolgt	Eintrag gehört nicht in die Referenz.
falsches Konzept, nicht im Brief, falscher Bezug	Eintrag gehört nicht in die Referenz.
zu allgemein, zu spezifisch, Dublette, unklar	*toleriert*: zählt später weder als Treffer noch als Fehler.
ergänzter fehlender Eintrag mit Code	Wird in die Referenz aufgenommen.
eingestreuter Fehler	Der Originaleintrag gilt.

13.4. Einfrieren und Auftauen

Referenz einfrieren … übernimmt die zusammengeführten Bewertungen als Referenz. Es klappt nur, wenn alle Briefe abgeschlossen und alle Abweichungen geklärt sind; sonst nennt die Meldung die offenen Briefe. Danach sind die Bewertungen unveränderlich.

Referenz auftauen … macht sie wieder änderbar, geht aber nur bei entsperrtem Datensatz. Bereits gelaufene Regressionsläufe beziehen sich dann auf die alte Referenz; nach erneutem Einfrieren sollte ein neuer Basis-Lauf angelegt werden.

Kapitel 14.

KI-Judges

14.1. Wozu

Ein KI-Judge bewertet alle Einträge eines Datensatzes nach denselben Regeln wie die Ärzte und nennt fehlende Einträge. Seine Urteile liegen getrennt von den ärztlichen Bewertungen; auch der Konsens-Prüfer sieht sie nicht. Er dient

- als **KI-Referenz** für Regressionstests, solange keine ärztliche Referenz vorliegt,
- als **zusätzlicher Bewerter**, der in der Studie gegen die Ärzte gemessen wird.

14.2. Verfügbare Judges

Im Reiter Referenz & Regression, Spalte KI-Judge, listet das Auswahlfeld alle konfigurierten Modelle, zum Beispiel:

Eintrag	Ort	Bemerkung
gpt-oss-120b (lokal)	Kliniknetz	Zugleich das Modell der Pipeline; siehe Abschnitt 14.7.
glm-52 (lokal)	Kliniknetz	Andere Modellfamilie als die Pipeline; gut als unabhängiger Judge.
claude-opus-5-5 (extern)	Anthropic-API	Briefe verlassen die Klinik. Nur mit Zustimmung des Datenschutzes.

14.3. Bewerten lassen

1. Judge im Auswahlfeld wählen.
2. Bewerten lassen … . Bei externen Judges zusätzlich `EXTERN` eintippen.
3. Hat der Judge schon Briefe bewertet, fragt das System: Nur fehlende (fehlende und fehlerhafte Briefe) oder Alle neu bewerten.
4. Der Fortschritt erscheint im Panel Hintergrundprozesse und oben im Statusfeld, etwa „läuft, 38 % (19 bewertet, 0 Fehler), bewertet gerade Brief 20 seit 29 s“.

Ein Brief dauert bei lokalen Denkmodellen typischerweise eine halbe bis wenige Minuten. Wird eine Antwort abgeschnitten, wiederholt der Judge den Brief einmal mit weniger Denkaufwand.

14.4. Beenden und aufräumen

Lauf beenden
Beendet den Lauf des gewählten Judge nach dem Brief, an dem er gerade arbeitet. Läuft der gewählte Judge nicht, bietet der Knopf an, alle Judge-Läufe des Datensatzes zu beenden. Läufe, deren Worker nicht mehr existiert, werden sofort beendet.

Fehlerprotokoll löschen ...
Entfernt fehlgeschlagene KI-Bewertungen (Verbindungsfehler, zurückgehaltene Briefe). Gültige Bewertungen bleiben geschützt.

14.5. KI-Referenz einfrieren

Wenn der gewählte Judge alle Briefe fehlerfrei bewertet hat: KI-Referenz einfrieren ... und mit dem Datensatznamen bestätigen. Die Statusanzeige zeigt dann „Quelle KI-Judge *Modell*“. Liegt später eine ärztliche Referenz vor: entsperren, auftauen, nach der ärztlichen Prüfung neu einfrieren und einen neuen Basis-Lauf anlegen.

14.6. Datenschutz bei externen Judges

Pseudonymisiert ist nicht anonymisiert

Arztbriefe enthalten in der Regel Namen, Geburtsdaten und Adressen. Vor dem Einsatz eines externen Judge müssen Datenschutz-Folgenabschätzung, Vertrag zur Auftragsverarbeitung und gegebenenfalls das Ethikvotum dies abdecken. Das Backend hält Briefe mit Geburtsdatum oder „Herr/Frau + Name“ zurück; das ist ein Sicherheitsnetz, keine Anonymisierung.

14.7. Methodische Einordnung

- Ein Judge, der dasselbe Modell ist wie die Pipeline, übersieht eher die eigenen Fehler. Für Regressionstests ist das vertretbar, weil sie Veränderungen messen; für die Studie einen Judge aus einer anderen Modellfamilie nehmen.
- Für eine Publikation ist der KI-Judge kein Goldstandard, sondern ein weiterer Bewerter, dessen Übereinstimmung mit den Ärzten berichtet wird (Kapitel 16).

Kapitel 15.

Regressionstests

15.1. Zwei Arten von Läufen

Knopf	Was passiert
Lauf starten (neu extrahieren) …	Alle Briefe gehen mit dem aktuellen Stand erneut durch die Pipeline, mit dem Modell, das im Hauptfenster unter *LLM Model* gewählt ist (ohne Deep Mode); die Ergebnisse werden mit der Referenz verglichen. Der Bestand wird dabei nicht verändert. Dauert etwa so lange wie entsprechend viele Extraktionen.
Gespeicherte Phenopackets prüfen	Vergleicht die im Bestand gespeicherten Phenopackets mit der Referenz, ohne neu zu extrahieren. Schnell; zeigt, wie gut der aktuelle Bestand ist. Briefe, deren Text sich im Bestand geändert hat, werden übersprungen.

Läufe setzen eine eingefrorene Referenz voraus. Pro Datensatz läuft höchstens ein Lauf gleichzeitig. Die Tabelle der Läufe zeigt in der Spalte *Modell*, mit welchem Sprachmodell extrahiert wurde (bei gespeicherten Phenopackets „Bestand“). Wie man zwei Modelle systematisch vergleicht, beschreibt Kapitel 17.

15.2. Einrichten: Basis und Grundrauschen

1. Lauf starten (neu extrahieren) … mit einer Bezeichnung wie „Basis Oktober 2026“.
2. Nach Abschluss den Lauf in der Tabelle markieren und Als Basis festlegen.
3. Ohne Änderung an der Pipeline sofort einen zweiten Lauf starten. Die Spalte *ΔF1 zur Basis* zeigt das Grundrauschen, denn ein Sprachmodell liefert nicht bei jedem Lauf exakt dasselbe.
4. Liegt das Rauschen über 2 Prozentpunkten, in der `.env` eine größere Toleranz setzen, etwa `ONTOTRIAL_REGRESSION_TOLERANCE=0.03`.

Basis immer neu extrahieren

Die gespeicherten Phenopackets können aus älteren Pipeline-Ständen stammen. Spätere Läufe extrahieren neu; die Basis sollte das deshalb auch tun.

15.3. Im Alltag

Nach jeder Änderung an Prompts, Intercepts, Filterregeln, Modell oder Code einen Lauf mit Lauf starten (neu extrahieren) … starten und die Spalte *Ergebnis* lesen:

ok
Kein Bereich hat sich über die Toleranz hinaus verschlechtert.

Regression
In mindestens einer Domäne oder insgesamt ist der F1-Wert stärker gefallen als die Toleranz. Der Detailbereich nennt die Domänen und listet je Brief, welche Einträge *verloren* gingen, welche *neu falsch* dazukamen und wie viele *neu richtig* sind.

Bericht als CSV liefert jeden Unterschied Brief für Brief: fehlend, zusätzlich, hierarchisch verwandt, Attributfehler.

15.4. Die Kennzahlen

Für jeden Brief werden Referenz und Extraktion je Domäne abgeglichen:

1. **Treffer (TP)**: gleicher Code. Bei mehreren Kandidaten wird der mit passender Seite, passendem Datum und Wert bevorzugt.
2. **Verwandt**: kein gleicher Code, aber hierarchisch verwandt (Ober- oder Unterbegriff). Zählt nur in der toleranten Wertung als Treffer.
3. **Fehlend (FN)**: Referenzeintrag ohne Gegenstück.
4. **Zusätzlich (FP)**: extrahierter Eintrag ohne Gegenstück. Treffer auf tolerierte Referenzeinträge (Dublette, unklar, zu allgemein, zu spezifisch) zählen weder als FP noch als TP.

Kennzahl	Definition
Precision	TP / (TP + FP + verwandt)
Recall	TP / (TP + FN + verwandt)
F1	harmonisches Mittel; „F1 tol.“ wertet Verwandte als Treffer
Seite, Datum, Status, Wert	Anteil korrekter Attribute unter den Treffern, bei denen die Referenz das Attribut prüfbar festlegt. Datum gilt als korrekt, wenn eines das andere als Präfix enthält (2021-03 und 2021-03-15). Messwerte werden numerisch verglichen.

15.5. Pipeline-Stand

Jeder Lauf speichert einen Fingerabdruck (Spalte *Pipeline-Stand*): Prüfsumme von `backend.pl`, Git-Stand, Extraktionsmodell des Laufs, Prüfsummen der Tabellen `llm_prompts`, `ontology_intercepts`, `filter_rules` und `catalog_versions` sowie die Mapping-Schalter aus der Umgebung. Zwei Läufe mit gleichem Fingerabdruck liefen mit derselben Pipeline.

Kapitel 16.

Validierungsstudie

16.1. Empfohlener Ablauf

1. Version der Pipeline einfrieren und Bestand damit neu extrahieren.
2. Pilotrunde mit 15 Briefen in einem eigenen Studien-Datensatz; Abweichungen besprechen und als Regeln festhalten.
3. Studien-Datensatz ziehen (Standard 150 Briefe, 30 % doppelt, 8 % eingestreute Fehler).
4. Prüfung durch die Ärzte, höchstens etwa 20 Briefe je Sitzung.
5. Konsens für alle Abweichungen.
6. Referenz einfrieren.
7. Optional: ein oder mehrere KI-Judges bewerten lassen.
8. Kennzahlen berechnen im Reiter Studie; Bewertungen als CSV für die eigene Statistik.

16.2. Die Kennzahlen im Reiter Studie

Kennzahl	Berechnung
Precision streng	Anteil der Einträge mit Konzept korrekt an allen bewerteten Einträgen (ohne Dubletten, unklare und eingestreute).
Precision tolerant	wie oben, „zu allgemein“ zählt als korrekt.
Seite, Datum, Status, Wert	Anteil korrekter Attribute unter den Einträgen mit korrektem Konzept; „nicht anwendbar“ zählt nicht mit.
Konzeptfehler	Anzahlen je Fehlertyp, dazu Dubletten und unklare Einträge.
Recall (Checkliste)	Anteil der Checklistenzeilen mit Abgleich „gefunden“ oder „Attributfehler“; „ohne Attributfehler“ nur „gefunden“. Je Kategorie und gesamt.
Übereinstimmung	Cohens κ und prozentuale Übereinstimmung der beiden Erstprüfer auf doppelt geprüften Briefen, für „korrekt ja/nein“, Konzept, Status, Seite und Datum.
Wachsamkeit	Anteil erkannter eingestreuter Fehler je Fehlertyp.

Kennzahl	Berechnung
KI-Judge vs. Ärzte	je Modell: Sensitivität für ärztlich gefundene Fehler, Spezifität, Vierfeldertafel, κ, Übereinstimmung bei fehlenden Einträgen, Erkennung eingestreuter Fehler.
Judges untereinander	κ der ersten beiden Modelle.

Konfidenzintervalle (95 %) stammen aus einem Cluster-Bootstrap über Briefe mit 2000 Ziehungen, weil Einträge eines Briefs nicht unabhängig sind. Briefe mit offener Abweichung gehen nicht in die Kennzahlen ein; ihre Zahl wird angegeben.

16.3. Rohdaten für die Statistik

Bewertungen als CSV liefert eine Zeile je Urteil (Semikolon-getrennt, UTF-8): Datensatz, Position, Pseudonym, Schicht, Doppelprüfung, Prüfer, Zeilentyp (Eintrag, ergänzt, Checkliste), Domäne, Code, Bezeichnung, Seite, Datum, verneint, Wert, eingestreuter Fehler sowie alle Urteilsfelder und Kommentare. Damit lassen sich alle Kennzahlen in R nachrechnen und weitere Analysen anschließen.

Berichten

Für Arbeiten mit Sprachmodellen empfiehlt sich die Darstellung nach TRIPOD-LLM. Pipeline-Stand, Modellversionen und Prompt-Version des Judge sind im System gespeichert und gehören ins Supplement.

Kapitel 17.

Zwei Sprachmodelle vergleichen

Dieses Kapitel beschreibt Schritt für Schritt, wie Sie zwei Extraktionsmodelle, etwa `gpt-oss-120b` und `glm-52`, an einem gemeinsamen Referenzstandard messen. Beide Modelle extrahieren dieselben Briefe; beide Ergebnisse werden mit derselben eingefrorenen Referenz verglichen.

17.1. Voraussetzungen

1. **Ein Datensatz mit eingefrorener Referenz.** Am besten eine ärztliche Referenz (Kapitel 12).
2. **Eine von beiden Modellen unabhängige Referenz.** Stammt die Referenz von einem KI-Judge, darf dieser Judge keines der beiden verglichenen Modelle sein. Eine KI-Referenz von gpt-oss-120b würde gpt-oss-120b bevorzugen, weil sie dessen Sicht der Briefe abbildet.
3. **Sonst gleiche Bedingungen.** Prompts, Intercepts, Filterregeln und Mapping-Schalter bleiben während des Vergleichs unverändert. Regressionsläufe extrahieren immer ohne *Deep Mode*, unabhängig von der Einstellung im Hauptfenster. Die Spalte *Pipeline-Stand* zeigt das: Läufe, die sich nur im Modell unterscheiden, haben bis auf das Modell denselben Fingerabdruck.
4. **Beide Modelle sind im Auswahlfeld *LLM Model*** im Hauptfenster verfügbar.

> **Nicht mit „Gespeicherte Phenopackets prüfen“ vergleichen**
>
> Die gespeicherten Phenopackets stammen von dem Modell, mit dem der Bestand einmal extrahiert wurde. Für einen Modellvergleich immer `Lauf starten (neu extrahieren) …` verwenden.

17.2. Schritt für Schritt

Schritt 1: Modell A wählen. Im Hauptfenster oben rechts unter *LLM Model* das erste Modell wählen, etwa `gpt-oss-120b`.

Schritt 2: Validierungsfenster öffnen. Kandidatenliste, Zahnrad-Menü, Validation…; den Datensatz wählen und zum Reiter Referenz & Regression wechseln.

Schritt 3: Ersten Lauf mit Modell A starten. Lauf starten (neu extrahieren) … . Der Dialog nennt das Modell aus dem Hauptfenster und schlägt es als Bezeichnung vor; ergänzen Sie „Lauf 1“, also etwa „gpt-oss-120b Lauf 1“. Fortschritt im Panel Hintergrundprozesse.

Schritt 4: Zweiten Lauf mit Modell A starten, sobald der erste fertig ist: „gpt-oss-120b Lauf 2“. Er misst, wie stark dasselbe Modell von Lauf zu Lauf schwankt.

Schritt 5: Modell B wählen. Im Hauptfenster *LLM Model* auf das zweite Modell umstellen, etwa `glm-52`.

Schritt 6: Zwei Läufe mit Modell B: „glm-52 Lauf 1“ und „glm-52 Lauf 2“.

Schritt 7: Vollständigkeit prüfen. In der Detailanzeige jedes Laufs darf keine Zeile „… Briefen ohne Ergebnis“ stehen. Fehlen Briefe, ist der Vergleich verzerrt; den Lauf dann wiederholen.

Schritt 8: Modell A als Basis. Lauf „gpt-oss-120b Lauf 1“ markieren und Als Basis festlegen .

Schritt 9: Ergebnisse ablesen. Die Tabelle zeigt für jeden Lauf F1, Precision und Recall gesamt, die Spalte *Modell* sowie *ΔF1 zur Basis*. Ein Klick auf einen Lauf zeigt im Detailbereich die Kennzahlen je Domäne (Diagnose, Befund, Eingriff, Medikation, Messwert) einschließlich der Genauigkeit von Seite, Datum, Status und Wert. Bei Läufen nach der Basis listet der Detailbereich außerdem je Brief, welche Einträge Modell B verloren hat und wo es neu falsch oder neu richtig liegt.

Schritt 10: Rauschen gegen Unterschied abwägen. Vergleichen Sie den Unterschied zwischen den Modellen mit dem Unterschied zwischen zwei Läufen desselben Modells.

Schritt 11: Berichte sichern. Für jeden Lauf Bericht als CSV laden und Sicherung laden für den gesamten Datensatz.

17.3. Ergebnisse festhalten

Eine Tabelle wie diese macht den Vergleich nachvollziehbar:

Lauf	F1	Precision	Recall	F1 tol.	Seite	Datum
gpt-oss-120b Lauf 1						
gpt-oss-120b Lauf 2						
glm-52 Lauf 1						
glm-52 Lauf 2						
Rauschen A (|Lauf 1 − Lauf 2|)						
Rauschen B						
Unterschied B − A (Mittel der Läufe)						

Faustregel. Ein Unterschied zwischen den Modellen ist erst dann bemerkenswert, wenn er deutlich größer ist als das Rauschen beider Modelle, also mindestens etwa doppelt so groß. Bei 50 Briefen sind Unterschiede von ein bis zwei Prozentpunkten in der Regel nicht belastbar.

17.4. Statistischer Vergleich in R

Für eine Aussage mit Konfidenzintervall vergleicht ein gepaarter Bootstrap über Briefe die beiden Läufe direkt. Die Ergebnisse je Brief liefert die API unter `/BBB/validation/runs/<Lauf-ID>`; die Lauf-ID steht in der ersten Spalte der Tabelle.

```
library(jsonlite)
basis <- "http://localhost:4007/BBB/validation/runs/"   # Adresse des Backends

lade_lauf <- function(id) {
  r <- fromJSON(paste0(basis, id), simplifyVector = FALSE)
  do.call(rbind, lapply(r$items, function(it) {
    res <- it$result
    if (is.null(res)) return(NULL)
    pa <- length(res$partial)            # hierarchisch verwandt
    data.frame(position = it$position,
               tp = length(res$tp_keys),
               fp = length(res$fp) + pa,
               fn = length(res$fn) + pa)
  }))
}

f1 <- function(d, s) {
  tp <- sum(d[[paste0("tp", s)]]); fp <- sum(d[[paste0("fp", s)]]); fn <- sum(d[[
    paste0("fn", s)]])
  2 * tp / (2 * tp + fp + fn)
}

a <- lade_lauf(12)   # Modell A
b <- lade_lauf(14)   # Modell B
d <- merge(a, b, by = "position", suffixes = c(".a", ".b"))   # nur Briefe mit
    Ergebnis in beiden Läufen

set.seed(1)
boot <- replicate(2000, { s <- d[sample(nrow(d), replace = TRUE), ]; f1(s, ".b") - f1
    (s, ".a") })
cat("F1 A:", round(f1(d, ".a"), 3), " F1 B:", round(f1(d, ".b"), 3),
    " Delta B-A:", round(f1(d, ".b") - f1(d, ".a"), 3),
    " 95%-KI:", round(quantile(boot, c(0.025, 0.975)), 3), "\n")
```

Schließt das Konfidenzintervall die Null nicht ein, unterscheiden sich die Modelle auf diesem Datensatz. Für eine Publikation den Vergleich mit beiden Läufen je Modell wiederholen und das Ergebnis zusammen mit dem Rauschen berichten.

17.5. Häufige Fallstricke

- **Referenz vom selben Modell.** Siehe Voraussetzungen; dann ist der Vergleich zugunsten dieses Modells verzerrt.

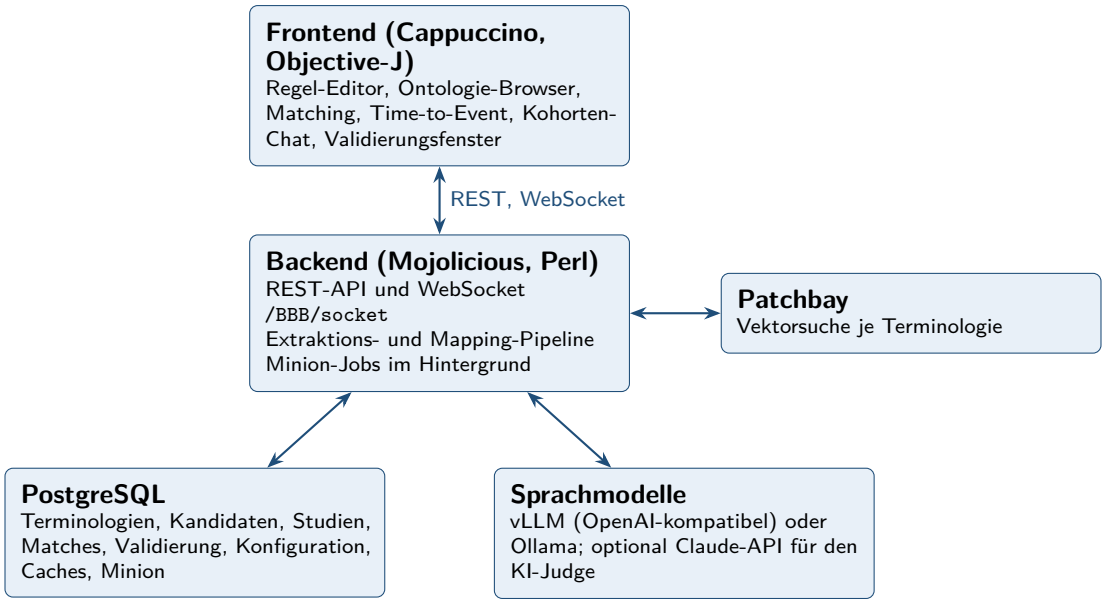
- **Geänderte Pipeline zwischen den Läufen.** Ein Update von Prompts oder Intercepts mitten im Vergleich vermischt zwei Effekte. Der Pipeline-Stand zeigt solche Änderungen.
- **Unterschiedliche Ausfälle.** Scheitern bei einem Modell mehr Briefe (Zeitüberschreitung, abgeschnittene Antwort), werden die Kennzahlen auf unterschiedlichen Briefen berechnet. Das R-Skript vergleicht deshalb nur Briefe, die in beiden Läufen ein Ergebnis haben.
- **Deep Mode.** Regressionsläufe verwenden ihn nicht. Ergebnisse aus Läufen sind deshalb nicht direkt mit Extraktionen vergleichbar, die im Hauptfenster mit Deep Mode erzeugt wurden.

Teil III.

Technische Dokumentation

Kapitel 18.

Architektur



Frontend
: Desktop-artige Weboberfläche nach dem Cocoa-MVC-Muster. Die Datenbindung (Fireside-ORM) hält ein Objekt je Datenbankzeile und übernimmt Änderungsmeldungen aus dem WebSocket. Die Oberfläche ist in `gui.gsmarkup` beschrieben; Werkzeuge liegen in `ToolsController.j` und `ValidationController.j`.

Backend
: `backend.pl` mit asynchroner REST- und WebSocket-API. Lang laufende Arbeiten (Briefimport, Neu-Extraktion, Regressionsläufe, KI-Judges) laufen als Minion-Jobs in einem eigenen Worker-Prozess. Änderungen werden per PostgreSQL LISTEN/NOTIFY an alle Browser verteilt.

Sprachmodelle
: Jedes OpenAI-kompatible vLLM oder Ollama. Modelle mit `-mlx` oder `ollama` im Namen werden an Ollama geleitet.

Patchbay
: Zustandslose Vektorsuche je Terminologie; Aufrufe werden in einer Warteschlange gedrosselt.

18.1. Dateien

Datei	Inhalt
`backend.pl`	gesamtes Backend
`migrations/ontotrial.sql`	Datenbankschema als Migration (Abschnitt 22)
`update_catalogs.pl`	Pflege der Terminologien
`.env`	Konfiguration und Schlüssel neben `backend.pl`
`public/Frontend/AppController.j`	Hauptanwendung
`public/Frontend/ToolsController.j`	Export, Propensity Matching, Neu-Extraktion
`public/Frontend/ValidationController.j`	Validierungsfenster
`public/Frontend/Resources/gui.gsmarkup`	Oberflächenbeschreibung, u. a. Aktionsmenü
`README.md`	Kurzdokumentation im Repository

Kapitel 19.

Extraktion und Mapping

19.1. Schritt 1: Zerlegung des Textes

Der Text wird bereinigt (HTML, KIS-Makros) und in Abschnitte zerlegt. Augenärztliche Abschnittsköpfe wie „Visus R:“, „VAA L:“ oder „Fundus BA:“ werden erkannt; die Seite wird als Vorgabe mitgegeben, und Befunde derselben Untersuchung werden nie zwischen Abschnitten getrennt.

Jeder Abschnitt geht mit einem strengen JSON-Schema an das Sprachmodell. Jede Zeile enthält Wortlaut, kanonischen Begriff, Datum, Ausschluss, Domäne, Seite, anatomische Lokalisation, Anamnese-Kennzeichen und gegebenenfalls Demografie. Danach greifen:

- **Filterregeln** aus `filter_rules` (Schutz von Pathologien und Eingriffen, Normalbefunde, nicht kodierbare Angaben).
- **Verankerung**: Morphologie ohne Organ („Narbe“, „Ödem“) wird um die Lokalisation ergänzt oder verworfen.
- **Korrekturen**: auf demselben Auge ausgeschlossene Verdachtsdiagnosen entfallen; Domänen werden korrigiert (Motilitätseinschränkung als HPO, „IOL in loco“ als Pseudophakie, konservative Tropftherapie nie als OPS); benannte Entzündungen werden doppelt als HPO und ICD-10 kodiert.

Im *Deep Mode* erstellt das Modell zuerst eine freie klinische Analyse mit Vortabelle, die zusammen mit dem Originaltext in die strukturierte Extraktion eingeht.

19.2. Schritt 2: Abbildung auf Terminologien

Stufe	Funktion
Intercepts	Kuratierte reguläre Ausdrücke in `ontology_intercepts` je Domäne und Priorität. Sie bilden auf einen Code ab oder unterdrücken einen Begriff. Codes werden gegen die Terminologie geprüft; die Bezeichnung kommt immer aus der Terminologie.
Übersetzung	Begriffe werden in die Indexsprache der Domäne übersetzt (Deutsch für ICD-10, OPS, ATC; Englisch für HPO, LOINC). Ergebnisse werden je Prompt-Version zwischengespeichert.

Stufe	Funktion
Vektorsuche	Patchbay liefert die nächsten Kandidaten (Prompts 51/52 HPO, 64 ICD-10, 65 OPS, 66 ATC, 69 LOINC).
Schutzregeln	Plausibles ID-Format, Distanzschwelle je Domäne, okulärer Bezug (augenbezogener Begriff nicht auf nicht-okulären Code und umgekehrt), Lokalisationsprüfung, veraltete Codes verworfen, ICD-10- und OPS-Codes müssen in den Tabellen existieren.
ICD-10-Besonderheiten	Anamnese-Codes Z85–Z87 gesperrt, Beobachtungscodes nur für Anamnese, S- und Z89-Codes nur bei passendem Wortlaut, Arzneimittelallergien als Z88.8 mit ATC-Verweis.
Bezeichnungsprüfung	Hohe Ähnlichkeit allein genügt nicht: Die Bezeichnung (oder ein HPO-Synonym, ATC-Handelsname) muss den Begriff weitgehend abdecken, sonst folgt das Reranking.
Reranking	Die besten Kandidaten und ihr gemeinsamer Oberbegriff gehen mit Bezeichnung, Oberbegriff und Indextext an das Sprachmodell; akzeptiert werden nur „exakt“ oder „allgemeiner“. Entscheidungen werden zwischengespeichert.

19.3. Augenheilkunde

- **Visus**: dezimal, Snellen und Metertafel-Brüche; Low Vision nach Schulze-Bonsel (Handbewegungen 0,005, Lichtschein 0,002, Fingerzählen 0,013, Nulla lux 0,000); „idem“ aus dem Aufnahmevisus desselben Auges; Lochblende getrennt.
- **Korrektur**: ohne (sc), eigene Brille, bestkorrigiert, jeweils auf den augenspezifischen LOINC-Code abgebildet.
- **Refraktion**: Sphäre, Zylinder, Achse, sphärisches Äquivalent.
- **Paarige Messwerte**: Augendruck („Tensio 14/16“), Hertel, BMO-Fläche als getrennte Messungen rechts und links.
- **Weitere Parameter**: Hornhautdicke, Endothelzelldichte, RNFL, GCL/IPL, Cup-to-Disc-Ratio, Lidspalte, Levatorfunktion, Tränenfilmaufrisszeit, Gesichtsfeld.
- **Kontext**: mehrdeutige Befunde werden über den Abschnitt aufgelöst, etwa Hornhaut- gegenüber Netzhautneovaskularisation oder okulärer gegenüber genitalem Herpes.

19.4. Aufbau der Phenopackets

Domäne	Feld im Phenopacket	Seite, Datum, Status
Befund (HPO)	`phenotypicFeatures[].type`	Seite in `modifiers[]`, Datum `onset.timestamp`, `excluded`
Diagnose (ICD-10)	`diseases[].term`	Seite `primarySite`, Datum `onset.timestamp`, `excluded`
Eingriff (OPS)	`procedures[].code`	Seite `bodySite`, Datum `performedTime`
Medikation (ATC)	`medicalActions[].treatment.agent`	Seite `treatment.bodySite`, Datum `treatment.performedTime`
Messwert (LOINC)	`measurements[].assay`	Wert `value.quantity`, Seite in `modifiers[]`, Datum `timeOfCollection.timestamp`

Seitencodes: `HP:0012834` rechts, `HP:0012835` links, `HP:0012832` beidseits; SNOMED-Qualifier werden ebenfalls erkannt.

Kapitel 20.

Matching und Ähnlichkeit

20.1. Studien-Matching

Jeder Patient wird zweimal geprüft: rechtes Auge als Studienauge, linkes als Partnerauge, und umgekehrt mit gespiegeltem Phenopacket. Jedes Kriterium wird mit Status `inclusion_met`, `inclusion_missing`, `exclusion_clear` oder `exclusion_violation` protokolliert. Codes werden hierarchisch geprüft (`is_subclass_of`); verneinte Befunde gelten als Beleg für das Fehlen.

20.2. Phenopacket-Distanz

Block	Gewicht	Methode
Demografie	0,15	Altersdifferenz (Skala 15 Jahre), Geschlecht
Diagnosen	0,25	Best-Match-Mittel über die Tiefe der ICD-10-Hierarchie
Phänotypen	0,20	Best-Match-Mittel mit Jaccard-Ähnlichkeit der HPO-Vorfahren
Prozeduren	0,15	Best-Match-Mittel über die OPS-Hierarchie
Medikation	0,10	Best-Match-Mittel über die ATC-Ebenen
Messwerte	0,15	letzter Wert je Parameter und Auge, skaliert (Visus in logMAR)

Lateralität. Je Code wird nur zwischen *einseitig* (nur rechts oder nur links), *beidseitig* (beidseits kodiert oder rechts und links genannt) und *unbekannt* unterschieden; rechts und links gelten als gleichwertig. Ähnlichkeitsfaktor 1 bei gleicher Klasse, 0,9 bei unbekannter Seite, 0,8 bei einseitig gegenüber beidseitig. Messwerte bleiben augenbezogen; die beiden Augen werden als ungeordnetes Paar verglichen, gerade (R–R, L–L) oder gekreuzt (R–L, L–R), einheitlich über alle Parameter. Es gewinnt die Zuordnung mit mehr gemeinsamen Parametern, dann die mit kleinerer Distanz.

Fehlende Blöcke werden ignoriert und als `coverage` berichtet. Darauf bauen die Suche nach k nächsten Nachbarn und das gierige 1:n-Fall-Kontroll-Matching mit Caliper auf.

Kapitel 21.

Validierung: Algorithmen

21.1. Stichprobe

Grundgesamtheit sind Briefe mit Text ($\geq$ 300 Zeichen) und Phenopacket, die in keinem anderen Validierungsdatensatz stehen, je Brieftext einmal (MD5), optional ein Brief je Pseudonym. Jeder Brief erhält eine Zufallszahl `md5(seed:id)`. Schichten sind Quelle und Jahr; die Stichprobengröße wird proportional nach dem Verfahren der größten Reste verteilt, innerhalb der Schicht entscheidet die Zufallszahl. Doppelprüfung und eingestreute Fehler werden ebenfalls deterministisch aus dem Seed bestimmt.

21.2. Einträge und Schlüssel

Das Phenopacket wird in Einträge zerlegt (Domäne, Code, Bezeichnung, Seite R/L/B, Datum JJJJ[-MM[-TT]], verneint, Wert, Einheit). Jeder Eintrag erhält als Schlüssel den MD5-Hash aus Domäne, Code, Seite, Datum, Status und Wert, bei Wiederholung mit laufender Nummer. Bewertungen beziehen sich auf diesen Schlüssel.

21.3. Zusammenführung und Referenz

Konsens vor Einzelprüfung; doppelt zu prüfende Briefe brauchen zwei abgeschlossene Prüfungen; zwei Prüfungen stimmen überein, wenn für jeden Eintrag Urteil, Konzept, Status, Seite, Datum, Wert und die richtigen Angaben gleich sind und die ergänzten Einträge (Code, Seite, Datum, verneint) übereinstimmen. Die Ableitung der Referenz zeigt Abschnitt 13.3. Die Referenz wird beim Einfrieren je Brief als JSON gespeichert und ist danach unveränderlich.

21.4. Vergleich einer Extraktion mit der Referenz

1. **Exakte Treffer**: Für jeden Referenzeintrag wird unter den Extraktionseinträgen derselben Domäne mit gleichem Code der beste gewählt. Bewertung: +4 gleicher Status, +2 gleiche Seite, +1 gleiches Datum, +1 gleicher Wert.

2. **Verwandte Codes**: Übrige Referenzeinträge werden mit hierarchisch verwandten Codes gepaart. HPO: Ober- oder Unterbegriff über `is_subclass_of`. ICD-10, OPS, ATC: einer ist Präfix des anderen (mindestens drei Zeichen). Messwerte: keine Verwandtschaft.

3. **Tolerierte Einträge** (Dublette, unklar, zu allgemein, zu spezifisch) neutralisieren einen exakt passenden Extraktionseintrag.
4. **Rest**: Referenz ohne Partner zählt als FN, Extraktion ohne Partner als FP.
5. **Attribute** auf exakten Treffern: Seite exakt; Datum korrekt, wenn eines Präfix des anderen ist; bei „überflüssig“ muss das Datum fehlen; Messwerte numerisch mit relativer Toleranz 10^{-6}, sonst Textvergleich; Status über `excluded`. Unbekannte richtige Angaben werden nicht geprüft.

Kennzahlen je Domäne und gesamt (Mikro-Mittel). Mit verwandten Paaren V:

$$\mathrm{P} = \frac{TP}{TP + FP + V}, \qquad \mathrm{R} = \frac{TP}{TP + FN + V},$$

$$F_1 = \frac{2\,TP}{2\,TP + FP + FN + 2V}, \qquad F_1^{\mathrm{tol}} = \frac{2(TP + V)}{2(TP + V) + FP + FN}.$$

Regressionsurteil. Gegenüber dem Basis-Lauf wird ΔF_1 je Domäne und gesamt berechnet. „Regression“, wenn ein Wert um mehr als `ONTOTRIAL_REGRESSION_TOLERANCE` (0,02) fällt. Je Brief werden verlorene Treffer, neue Fehltreffer und neue Treffer gelistet.

Pipeline-Stand. MD5 von `backend.pl`, Git-Stand, Extraktionsmodell des Laufs, Prüfsummen von `llm_prompts`, `ontology_intercepts`, `filter_rules` und `catalog_versions` sowie die Mapping-Schalter; daraus ein 12-stelliger Hash.

21.5. Studienkennzahlen

Precision und Attributgenauigkeit aus den zusammengeführten Bewertungen, Recall aus dem Abgleich der Checkliste. Konfidenzintervalle: Cluster-Bootstrap über Briefe, 2000 Ziehungen mit festem Startwert, Perzentile 2,5 und 97,5 (nur bei mindestens 20 gültigen Ziehungen). Cohens $\kappa = (p_o - p_e)/(1 - p_e)$ für die beiden ersten Prüfer. Wachsamkeit: Seitentausch gilt als erkannt bei Seite „falsch“ oder „fehlt“ bzw. Konzept nicht korrekt, Datumsverschiebung bei Datum „falsch“ bzw. Konzept nicht korrekt, entfernter Eintrag, wenn ein ergänzter Eintrag denselben Code hat.

21.6. KI-Judge

Der Judge erhält einen festen Systemprompt mit den Regeln des Bewertungshandbuchs (Prompt-Version = gekürzter MD5) und je Brief Briefdatum, Brieftext und nummerierte Einträge E1, E2, … Die Antwort ist per JSON-Schema erzwungen (Claude: Structured Outputs; vLLM: `response_format` mit `json_schema`; Ollama: `format`). Vorgeschlagene Codes für fehlende Einträge werden normalisiert und gegen die Terminologie geprüft; unbekannte werden nicht übernommen. Lokale Modelle: `reasoning_effort medium`; bei abgeschnittener Antwort ein zweiter Versuch mit `low`; Verbindungsfehler werden bis zu dreimal (lokal) bzw. fünfmal (extern) wiederholt. Ergebnisse liegen in `validation_ai_reviews` je Brief und Modell.

Kapitel 22.

Datenbank

22.1. Schema und Migrationen

Das Schema steht in `migrations/ontotrial.sql` und wird beim Start von `backend.pl` als Mojo::Pg-Migration mit dem Namen `ontotrial` eingespielt. Eine Datenbank, die das Kernschema schon hat, wird beim ersten Start nur als Version 1 eingetragen.

Version	Inhalt
1	Kernschema: Tabellen, View `patients`, Funktionen (`is_subclass_of` u. a.), Indizes, Erweiterungen `pg_trgm` und `fuzzystrmatch`
2	Validierung: Datensätze, Briefe, Bewertungen, Läufe, KI-Bewertungen; Schutz-Trigger
3	Fehlerhafte KI-Bewertungen dürfen auch in gesperrten Datensätzen gelöscht werden

Neue Änderungen als `--4up` / `--4down` anhängen; ausgerollte Versionen nie ändern. Minion verwaltet seine Tabellen selbst; `update_catalogs.pl` legt `catalog_versions` beim ersten Import an.

22.2. Tabellen

Gruppe	Tabellen
Terminologien	`terms`, `isas`, `synonyms`, `xrefs`, `hpo_closure` (HPO); `icd10_terms`, `ops_terms`, `atc_terms`, `loinc_terms`; `catalog_versions`
Klinische Daten	`candidates` (Briefe, Phenopackets, Tags), `trials` (FHIR-Gruppen), `matches` (Ergebnisse und Protokolle); View `patients`
Konfiguration	`llm_prompts`, `ontology_intercepts`, `ontology_intercepts_patchlog`, `filter_rules`
Caches	`translation_cache`, `mapping_rerank_cache`
Validierung	`validation_sets`, `validation_items`, `validation_reviews`, `validation_runs`, `validation_run_items`, `validation_ai_reviews`
Betrieb	Minion-Tabellen, `mojo_migrations`

22.3. Schutz der Validierungsdaten

Trigger verhindern: Löschen gesperrter Datensätze; Ändern von Brieftext, Schnappschuss, Einträgen und eingestreuten Fehlern; Ändern der Referenz und aller Bewertungen nach dem Einfrieren; Auftauen gesperrter Datensätze; Löschen gültiger KI-Bewertungen gesperrter Datensätze. Die generischen Tabellen-Routen `/BBB/validation_*` sind gesperrt.

Kapitel 23.

Konfiguration

Die Konfiguration steht als Umgebungsvariablen in der Datei `.env` neben `backend.pl`; das Backend liest sie beim Start. Bereits gesetzte Umgebungsvariablen haben Vorrang. Die Datei gehört nicht ins Repository und sollte die Rechte 600 haben.

23.1. Sprachmodelle und Dienste

Variable	Bedeutung
`LLM_PROVIDER`	`vllm` oder `ollama`
`VLLM_ENDPOINT`, `VLLM_API_KEY`, `VLLM_MODEL`	Endpunkt, Schlüssel und Standardmodell für vLLM
`OLLAMA_ENDPOINT`, `OLLAMA_MODEL`	Endpunkt und Modell für Ollama
`PATCHBAY_URL`	Adresse der Vektorsuche

23.2. Qualität des Mappings

Variable	Standard	Wirkung
`ONTOTRIAL_RERANK`	1	Reranking der Kandidaten
`ONTOTRIAL_RERANK_ACCEPT_SIM`	0,985	Ähnlichkeit, ab der ohne Reranking akzeptiert wird
`ONTOTRIAL_RERANK_TOP_K`	5	Kandidaten im Reranking
`ONTOTRIAL_OCULAR_PRIOR`	1	nicht-okuläre Codes für okuläre Begriffe ablehnen
`ONTOTRIAL_OCULAR_PRIOR_MIN_SIM`	0,98	Ähnlichkeit, die diese Regel aufhebt
`ONTOTRIAL_ICD_OCULAR_PRIOR`	1	dieselbe Regel für ICD-10
`ONTOTRIAL_TRANSLATION_CACHE`	1	Übersetzungen zwischenspeichern
`ONTOTRIAL_LABEL_GATE_MIN_COVERAGE`	0,67	Anteil der Wörter, den die Bezeichnung abdecken muss
`ONTOTRIAL_ICD_REQUIRE_RESOLVED`	1	ICD-10-Code muss existieren

Variable	Standard	Wirkung
ONTOTRIAL_BLOCK_HISTORY_ZCODES	1	Z85–Z87 sperren
ONTOTRIAL_INTERCEPT_VALIDATE	1	Intercepts mit unbekannten Codes ignorieren
ONTOTRIAL_PARENT_CANDIDATE	1	gemeinsamen Oberbegriff im Reranking anbieten
ONTOTRIAL_GROUND_GENERIC	1	Lokalisation an Begriffe ohne Organ anhängen

23.3. Import, Export, Validierung und KI-Judges

Variable	Bedeutung (Standard)
ONTOTRIAL_IMPORT_DATE_TOLERANCE_DAYS	Tage um das Briefdatum, in denen ein Import als derselbe Tag gilt (0)
ONTOTRIAL_EXPORT_TOKEN	Token für den Long-Format-Export (nicht gesetzt)
ONTOTRIAL_REGRESSION_TOLERANCE	zulässiger F1-Abfall (0,02)
ONTOTRIAL_JUDGE_LOCAL_MODELS	lokale Judges, kommagetrennt (Pipeline-Modell)
ONTOTRIAL_JUDGE_URL_<MODELL>, ONTOTRIAL_JUDGE_KEY_<MODELL>	eigener Endpunkt bzw. Schlüssel je Modell; Name in Großbuchstaben, Sonderzeichen als _ (Endpunkt und Schlüssel der Pipeline)
ONTOTRIAL_JUDGE_REASONING_EFFORT	low, medium, high (medium)
ONTOTRIAL_JUDGE_LOCAL_MAX_TOKENS	Ausgabe-Limit lokaler Judges (32000)
ONTOTRIAL_JUDGE_LOCAL_TEMPERATURE	Temperatur lokaler Judges (Modell-Standard)
ONTOTRIAL_JUDGE_LOCAL_THINKING	0 schaltet den Denkmodus über `chat_template_kwargs` ab
ONTOTRIAL_JUDGE_LOCAL_PII_GUARD	Namenssperre auch für lokale Judges (0)
ANTHROPIC_API_KEY	Schlüssel für den Claude-Judge
ONTOTRIAL_JUDGE_MODEL	Claude-Modell (`claude-opus-5-5`)
ONTOTRIAL_JUDGE_MAX_TOKENS, ONTOTRIAL_JUDGE_TEMPERATURE	Ausgabe-Limit (16000) und Temperatur (Modell-Standard) des Claude-Judge
ONTOTRIAL_JUDGE_PII_GUARD, ONTOTRIAL_JUDGE_PII_REGEX	Namenssperre für externe Judges (1) und eigene Regel
ONTOTRIAL_JUDGE_URL	abweichender Endpunkt für Claude
HTTPS_PROXY, SSL_CERT_FILE	Proxy und Zertifikatsdatei für externe Verbindungen

Kapitel 24.

Betrieb

24.1. Installation

```
cpanm Mojolicious Mojo::Pg Minion DateTime
createdb hpo
perl backend.pl daemon -l http://*:4007      # Entwicklung; legt das Schema an
```

Danach Terminologien laden (Abschnitt 24.4) und die Prompts in `llm_prompts` eintragen:

`atomic_criteria_extraction`	Extraktion, Schritt 1
`atomic_criteria_reasoning`	Analyse im Deep Mode
`criterion_retrieval_translator`	Übersetzung für die Vektorsuche
`mapping_candidate_reranker`	Reranking
`feasibility_chat_assistant`	Kohorten-Chat

24.2. Prozesse und Neustarts

```
hypnotoad backend.pl            # Webserver starten oder Hot Deploy
hypnotoad -s backend.pl         # Webserver stoppen
perl backend.pl minion worker   # Worker für Hintergrundjobs
```

Änderung	Neu starten
nur Frontend-Dateien	nichts; Browser mit Cmd+Shift+R neu laden
`backend.pl`, Routen und Anzeige	Hypnotoad (Hot Deploy genügt)
`backend.pl`, Hintergrundjobs	Hypnotoad *und* Worker
`.env`	Hypnotoad stoppen und starten *und* Worker
`migrations/ontotrial.sql`	Hypnotoad; die Migration läuft beim Start

Zwei Fallstricke

Ein Worker-Neustart bricht laufende Jobs ab; vorher warten oder danach mit Lauf beenden aufräumen. Ein Hot Deploy übernimmt die Umgebung des alten Prozesses; geänderte Werte in der `.env` wirken erst nach vollständigem Stopp und Start.

24.3. Nützliche Befehle

```
perl backend.pl minion job -w                                    # aktive Worker
perl backend.pl minion job -t validation_judge_task -l 5         # Judge-Jobs
perl backend.pl minion job -t validation_run_task -l 5           # Regressionsläufe
perl backend.pl minion job 670428                                # Details eines Jobs
psql hpo -c "SELECT * FROM mojo_migrations ORDER BY name"        # Schema-Version
```

24.4. Terminologien aktualisieren

```
perl update_catalogs.pl inventory
perl update_catalogs.pl hpo hp.obo --dry-run
perl update_catalogs.pl icd10 <claml.xml> --kodes <kodes.txt> --dry-run
perl update_catalogs.pl ops   <claml.xml> --kodes <kodes.txt> --dry-run
perl update_catalogs.pl atc   <datei.csv> --version <jahr> --dry-run
perl update_catalogs.pl check backend.pl
```

Reihenfolge: Bestand prüfen, Quellen laden, jeden Import erst mit `--dry-run`, dann echt; LOINC über `/_import_loinc`; Hypnotoad und Worker neu starten; Patchbay-Indizes *nach* dem Import neu bauen; `check` ausführen, betroffene Studienkriterien korrigieren, Kandidaten neu extrahieren und Matching wiederholen. Jeder Import legt eine Sicherung `<tabelle>_bak_<zeitstempel>` an und wird in `catalog_versions` protokolliert.

> **Validierung nach Updates**
>
> Nach jedem Katalog-Update einen Regressionslauf starten. Der Pipeline-Stand ändert sich über die Prüfsumme von `catalog_versions`.

Kapitel 25.

API-Referenz

Alle Routen liegen unter `/BBB`. Die API hat keine eigene Anmeldung und sendet `Access-Control-Allow-Origin:*`; das Backend gehört in ein geschütztes Netzsegment hinter einen authentifizierenden Reverse Proxy.

25.1. Extraktion

Art	Route	Funktion
POST	`/BBB/extract_fhir_inex_criteria`	Studientext zu FHIR R6 Group
POST	`/BBB/extract_phenopacket`	Text zu Phenopacket; mit `candidate_id` gespeichert und verteilt
POST	`/BBB/extract_phenopacket_from_letter`	wie oben, JSON oder Text, mit `deep_mode`
POST	`/BBB/import_and_extract_letter`	Brief speichern und Extraktion als Job einreihen
POST	`/BBB/candidates/recompute_by_tag`	alle Kandidaten mit Tag neu extrahieren
POST	`/BBB/resolve_term`	einen Begriff auf einen Code abbilden

25.2. Matching, Analyse und Export

Art	Route	Funktion
POST	`/BBB/run_all_matches`	Studien gegen Kandidaten; optional `trial_id`, `candidate_id`, `tag`
GET	`/BBB/matches`	Ergebnisse mit Status
POST, GET	`/BBB/trials/:id/time_to_eligibility`, `.csv`	Zeit bis Eignung als JSON oder CSV
POST	`/BBB/phenopacket_distance`	Distanz zweier Kandidaten (`a`, `b`, `weights`, `exact`, `explain`)

Art	Route	Funktion
POST	`/BBB/phenopacket_distance/nearest`	k nächste Nachbarn
POST	`/BBB/propensity_match`	Fall-Kontroll-Matching
POST	`/BBB/export/anonymized_phenopackets`	anonymisierter Export (`tag` oder `candidate_ids`, $k \geq 5$)
GET	`/BBB/export/long/events`, `/BBB/export/long/patients`	Long-Format als CSV oder NDJSON
POST	`/BBB/chat/query`, `/BBB/chat/execute_sql`	Kohorten-Chat und SQL
POST	`/BBB/candidates/batch_tag`	Tags setzen
GET	`/BBB/<ontologie>/roots`, `children/:id`, `search/:q`	Terminologie-Browser für hpo, icd10, ops, atc, loinc

Long-Format-Export. Wichtige Parameter: `has_tag` (Population), `has_code` (Patienten mit Code oder Unterbegriff), `tag` mit `tag_scope`, `from`/`to` (Briefdatum), `event_from`/`event_to`, `domains`, `sources`, `include_excluded`, `dedup`, `undated`, `format`, `token`. Spalten der Ereignisse: Pseudonym, Paket, Briefdatum, Domäne, Code, Bezeichnung, Seite (`site_id`), Datum wie gespeichert (`date_raw`), verneint, Wert, Einheit, Operator, Textwert, Anzahl Nennungen, letzter Brief.

```
url <- "https://<host>/BBB/export/long/events?has_code=OPS:5-125"
f <- tempfile(fileext = ".csv")
download.file(url, f, mode = "wb", quiet = TRUE,
              headers = c("X-Export-Token" = Sys.getenv("ONTOTRIAL_EXPORT_TOKEN")))
ereignisse <- read.csv(f, fileEncoding = "UTF-8", na.strings = c("", "NA", "NULL"))
```

25.3. Validierung

Art	Route	Funktion
GET, POST	`/BBB/validation/sets`	Datensätze listen bzw. ziehen (`confirm:"ANLEGEN"`)
GET	`/BBB/validation/sets/:id/items`	Briefe mit Stand je Prüfer
GET	`/BBB/validation/items/:id`	Brief, Einträge, eigene Bewertungen
POST	`/BBB/validation/items/:id/review`	Bewertungen speichern
GET	`/BBB/validation/sets/:id/status`	offene Briefe und Abweichungen
POST	`/BBB/validation/sets/:id/freeze`, `unfreeze`, `lock`	Referenz einfrieren (`source:"judge"`, `model`), auftauen, sperren
DELETE	`/BBB/validation/sets/:id`	Datensatz löschen (nur entsperrt)
POST, GET	`/BBB/validation/sets/:id/runs`	Lauf starten (`mode`, `model`, `label`) bzw. listen

Art	Route	Funktion
GET	`/BBB/validation/runs/:id`, `report.csv`	Lauf mit Ergebnissen je Brief, CSV-Bericht
POST	`/BBB/validation/sets/:id/baseline`	Basis-Lauf festlegen
POST, GET	`/BBB/validation/sets/:id/judge`	KI-Judge starten (`provider`) bzw. Status
POST	`/BBB/validation/sets/:id/judge/stop`, `judge/clear_errors`	Judge beenden, Fehlerprotokoll löschen
GET	`/BBB/validation/sets/:id/study`, `judgments.csv`, `export`	Studienkennzahlen, Rohdaten, Sicherung

25.4. Live-Meldungen

Der WebSocket `/BBB/socket` sendet Tabellenänderungen (`table`, `pk`, `type`, `data`; Inhalte über 7500 Zeichen als `truncated`) und Fortschrittsmeldungen `TASK_PROGRESS` mit `task_id`, `phase`, `progress`, `message`. Validierungsjobs melden sich als `validation_judge_<Datensatz>` und `validation_run_<Lauf>`.

Kapitel 26.

Sicherheit und Datenschutz

- **Netz**: keine eingebaute Anmeldung; generische Tabellenrouten, `/BBB/chat/execute_sql` und der Long-Format-Export können personenbezogene Daten lesen. Nur im geschützten Netz hinter einem authentifizierenden Proxy betreiben; für den Long-Format-Export `ONTOTRIAL_EXPORT_TOKEN` setzen (wird im Log maskiert).
- **Daten in der Datenbank**: `candidates` enthält Pseudonyme, Brieftexte und Phenopackets. Wer darauf Zugriff hat, kann exportierte Pakete über den Inhalt zuordnen.
- **Weitergabe**: nur über den anonymisierten Export; über die Anonymität für einen Empfänger entscheidet der Datenschutz.
- **Externe Sprachmodelle**: nur mit Zustimmung von Datenschutz und ggf. Ethikkommission; die Namenssperre ist ein Sicherheitsnetz, keine Anonymisierung.
- **Schlüssel**: in der `.env` mit Rechten 600, nie im Repository.

Kapitel 27.

Fehlerbehebung

Beobachtung	Ursache und Abhilfe
Klicks in einem Fenster kommen nicht an, Tastatur geht	Ein schwebendes Panel liegt unter dem Mauszeiger auf höherer Ebene. Betroffene Fenster als schwebendes Panel öffnen (`setFloatingPanel:YES`) oder das Panel schließen.
Lauf bleibt bei „wartet auf den Minion-Worker“	Kein oder ein alter Worker. Worker starten bzw. neu starten.
Job sofort „failed“ mit „Unbekannter Anbieter“	Worker läuft mit altem Code. Worker neu starten.
„läuft“, aber kein Fortschritt; Worker beendet	Lauf hängt; Lauf beenden räumt auf.
„Antwort abgeschnitten“	Ausgabe-Limit erschöpft. `ONTOTRIAL_JUDGE_REASONING_EFFORT=low` oder höheres Limit.
HTTP 400 mit „maximum context length“	Kontextfenster des Modells zu klein für das Ausgabe-Limit; `ONTOTRIAL_JUDGE_LOCAL_MAX_TOKENS` senken.
„SSL connect attempt failed“	Proxy (`HTTPS_PROXY`), Firewall-Zertifikat (`SSL_CERT_FILE`) oder veraltete Perl-SSL-Module (`cpanmIO::Socket::SSLNet::SSLeayMozilla::CA`).
„credit balance is too low“	Guthaben beim externen Anbieter aufladen.
Neues Modell fehlt im Judge-Auswahlfeld	`.env` geändert ohne vollständigen Neustart von Hypnotoad.
Änderungen an Frontend-Dateien wirken nicht	Browser-Cache: Cmd+Shift+R.
Patch meldet „Reversed (or previously applied)“	Patch ist schon eingespielt; nicht mit `y` antworten, sondern mit `patch -N` überspringen.
`.env`-Werte werden ignoriert	Variable in der Shell bereits gesetzt; neues Terminal oder `unset`.

Anhang A.

Kurzreferenz für Prüferinnen und Prüfer

Ablauf je Brief

1. Brief lesen. *Studie:* Checkliste ausfüllen, Checkliste abschließen, Extraktion zeigen.
2. Jeden Eintrag: Korrekt, Fehler … (mit richtiger Angabe), Dublette oder Unklar (mit Kommentar).
3. Fehlendes mit Fehlenden Eintrag ergänzen …, möglichst mit Code.
4. *Studie:* jede Checklistenzeile Gefunden, Attributfehler oder Fehlt.
5. Brief abschließen.

Fehlertypen

Konzept	korrekt · zu allgemein · zu spezifisch · falsches Konzept · nicht im Brief · falscher Bezug
Status	korrekt · Verneinung falsch · Verdacht als gesichert · Geplantes als erfolgt
Seite, Datum	korrekt · falsch · fehlt · überflüssig · nicht anwendbar *(+ richtige Angabe)*
Messwert	korrekt · falsch · nicht anwendbar *(+ richtiger Wert)*

Die fünf wichtigsten Regeln

1. Nur der Brieftext zählt.
2. Verdacht und Geplantes sind nicht gesichert bzw. nicht erfolgt.
3. Anamnestisches ohne Datum trägt kein Briefdatum.
4. „18/21“ heißt RA 18, LA 21.
5. Im Zweifel Unklar mit Kommentar, nicht raten.